

Tài Liệu Cấu Trúc Dữ Liệu & Giải Thuật

Học phần: SW102DV01 | Giảng viên: Vũ Đình Khôi

Hệ thống hóa nội dung Slide môn học

2026-07-07

Note

Tài liệu này được biên soạn và tổng hợp chi tiết từ 10 slide bài giảng môn học **Cấu trúc dữ liệu & Giải thuật (SW102DV01)** của giảng viên **Vũ Đình Khôi** tại Đại học Hoa Sen. Nội dung bao gồm lý thuyết cốt lõi, công thức đánh giá độ phức tạp thuật toán và mã nguồn C minh họa chuẩn hóa.

Tổng Quan về Cấu Trúc Dữ Liệu & Giải Thuật

Các Khái Niệm Cơ Bản

Kiểu Dữ Liệu (Data Type)

Kiểu dữ liệu là một ràng buộc lên các giá trị mà một biến có thể lưu giữ, đồng thời định nghĩa các phép toán có thể thực hiện trên các giá trị đó. * **Kiểu dữ liệu cơ bản (Primitive Data Types)**: Các kiểu được ngôn ngữ lập trình cung cấp sẵn (ví dụ: `int`, `char`, `float`, `double`, `boolean`). Dung lượng bộ nhớ được cấp phát tùy thuộc vào ngôn ngữ và kiến trúc hệ thống. * **Kiểu dữ liệu do người dùng định nghĩa (User-defined Data Types)**: Được xây dựng bằng cách kết hợp các kiểu cơ bản, ví dụ: `struct` trong C hoặc `class` trong C++/Java.

Kiểu Dữ Liệu Trừu Tượng (Abstract Data Type - ADT)

ADT là một mô hình toán học đi kèm với một tập hợp các phép toán được định nghĩa trên mô hình đó, nhưng **tách biệt hoàn toàn** khỏi cách thức cài đặt chi tiết (implementation). * *Ví dụ về kiểu tọa độ điểm*: Người dùng chỉ cần biết cấu trúc `Point` và các giao tiếp (interface) như tính khoảng cách Euclid mà không cần quan tâm bên trong lưu trữ bằng hệ tọa độ nào. * *Ví dụ về Chuỗi trong C (`string.h`)*: Không phải là một ADT thực sự vì người dùng biết rõ nó

được biểu diễn bằng mảng ký tự kết thúc bằng \0 và có thể truy cập trực tiếp qua con trỏ hoặc chỉ mục.

Cấu Trúc Dữ Liệu (Data Structure)

Cấu trúc dữ liệu là sự cài đặt cụ thể của một ADT trong bộ nhớ máy tính. Việc lựa chọn cấu trúc dữ liệu cần tối ưu hóa giữa hai yếu tố: **bộ nhớ (space)** và **thời gian xử lý (time)**. * Ví dụ: ADT Stack định nghĩa các hành động **push**, **pop**, **isEmpty** theo nguyên lý LIFO. Cấu trúc dữ liệu Stack có thể được cài đặt cụ thể bằng mảng tuyến tính (Array Stack) hoặc danh sách liên kết (Linked List Stack).

Giải Thuật (Algorithm)

Giải thuật là một dãy hữu hạn các bước rõ ràng nhằm hoàn thành một công việc thực tế hoặc giải quyết một bài toán trên máy tính.

Các Đặc Tính của Giải Thuật

1. **Tính khả thi (Feasibility):** Các bước trong thuật toán phải đủ đơn giản để có thể viết mã và thực thi.
2. **Tính hữu hạn (Finiteness):** Thuật toán phải dừng lại sau một số bước hữu hạn và thời gian hữu hạn.
3. **Tính xác định (Definiteness):** Các bước phải rõ ràng, không gây mơ hồ. Cùng một đầu vào (input) phải cho ra cùng một đầu ra (output).
4. **Tính hiệu quả (Efficiency):** Thực thi tối ưu về mặt thời gian và không gian bộ nhớ.

Các Phương Pháp Thiết Kế Thuật Toán Chính

- **Giải thuật tham lam (Greedy approach):** Lựa chọn phương án tối ưu cục bộ tại mỗi bước với hy vọng đạt được tối ưu toàn cục. Ví dụ: Bài toán đổi tiền.
- **Chia để trị (Divide and Conquer):** Chia bài toán lớn thành các bài toán con cùng loại, giải các bài toán con rồi tổng hợp kết quả. Ví dụ: Tìm kiếm nhị phân, Fake coin.
- **Quay lui (Backtracking):** Thử sai và tìm lời giải từng bước, loại bỏ các lựa chọn không dẫn tới lời giải. Thường kết hợp đệ quy. Ví dụ: Bài toán 8 quân hậu, Sudoku.
- **Quy hoạch động (Dynamic programming):** Chia nhỏ bài toán, lưu trữ kết quả của các bài toán con đã giải để tái sử dụng thay vì tính toán lại.

Phân Tích Độ Phức Tạp của Giải Thuật

Độ Phức Tạp Thời Gian (Time Complexity)

Thời gian ở đây không đo bằng giây (vì phụ thuộc vào phần cứng) mà ước lượng bằng **số lượng các phép toán cơ bản** (phép gán, phép so sánh, phép tính số học) mà thuật toán thực hiện dựa trên kích thước đầu vào n . * **Trường hợp tốt nhất (Best case)**: Thời gian thực thi ít nhất. * **Trường hợp xấu nhất (Worst case)**: Thời gian thực thi nhiều nhất. Thường được chọn làm tiêu chuẩn đánh giá. * **Trường hợp trung bình (Average case)**: Kỳ vọng toán học dựa trên xác suất phân phối đầu vào.

Ký Pháp Big O (O)

Ký pháp Big O dùng để mô tả giới hạn trên (tiệm cận trên) của tốc độ tăng của một hàm số khi kích thước đầu vào n tiến ra vô cùng:

$$T(n) = O(f(n)) \iff \exists c > 0, n_0 > 0 \text{ sao cho } T(n) \leq c \cdot f(n), \forall n \geq n_0$$

Các Bậc Tăng Trưởng Phổ Biến (Rates of Growth)

Xếp hạng hiệu suất từ tốt nhất đến tệ nhất:

Ký hiệu	Tên gọi	Ví dụ
$O(1)$	Hằng số (Constant)	Truy cập phần tử mảng qua chỉ mục
$O(\log n)$	Lôgarit (Logarithmic)	Tìm kiếm nhị phân
$O(n)$	Tuyến tính (Linear)	Tìm kiếm tuần tự
$O(n \log n)$	Tuyến tính-lôgarit	Merge sort, Quick sort
$O(n^2)$	Bình phương (Quadratic)	Bubble sort, Selection sort
$O(2^n)$	Mũ (Exponential)	Tháp Hà Nội, Fibonacci đệ quy
$O(n!)$	Giai thừa (Factorial)	Liệt kê hoán vị

Đệ Quy (Recursion)

Khái Niệm & Nguyên Lý Stack

Đệ quy là phương pháp giải quyết bài toán bằng cách gọi lại chính nó với quy mô nhỏ hơn, cho đến khi chạm tới **trường hợp cơ sở (base case)** để kết thúc.

Mỗi khi một hàm đệ quy được gọi, hệ thống sẽ đẩy một **activation record** chứa các tham số và biến cục bộ vào bộ nhớ **Stack**. Khi gặp điều kiện dừng, các lời gọi hàm sẽ lần lượt được lấy ra khỏi Stack (LIFO) để trả về giá trị.

Warning

Nếu thiếu điều kiện dừng hoặc điều kiện dừng không bao giờ đạt được, chương trình sẽ rơi vào lặp vô hạn và gây ra lỗi **Tràn bộ nhớ Stack (Stack Overflow)**.

Phân Loại Đệ Quy

1. **Đệ quy tuyến tính (Linear Recursion):** Thân hàm chứa duy nhất một lời gọi đệ quy. Ví dụ: *Tính giai thừa*.
2. **Đệ quy nhị phân/cây (Binary/Tree Recursion):** Thân hàm chứa hai lời gọi đệ quy. Ví dụ: *Dãy Fibonacci*.
3. **Đệ quy hỗ tương (Mutual Recursion):** Hàm A gọi hàm B và hàm B gọi lại hàm A.
4. **Đệ quy phi tuyến/đa tuyến (Multiple Recursion):** Lời gọi đệ quy nằm trong vòng lặp. Ví dụ: *Tháp Hà Nội*.
5. **Đệ quy đuôi (Tail Recursion):** Lời gọi đệ quy nằm ở cuối hàm, kết quả trả về trực tiếp không cần thực hiện thêm phép tính nào sau đó.

Các Bài Toán Đệ Quy Điển Hình (Mã nguồn C)

```
#include <stdio.h>

// 1. Tính giai thừa (Đệ quy tuyến tính) - O(n)
int factorial(int n) {
    if (n <= 1) return 1; // Base case
    return n * factorial(n - 1);
}

// 2. Tính lũy thừa a^n - O(n)
double power(double a, int n) {
    if (n == 0) return 1;
    if (n < 0) return 1.0 / power(a, -n);
}
```

```

    return a * power(a, n - 1);
}

// 3. Ước số chung lớn nhất (Euclid) -  $O(\log(\min(a,b)))$ 
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

// 4. Fibonacci (Đệ quy nhị phân) -  $O(1.618^n)$ 
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

// 5. Tháp Hà Nội (Đệ quy phi tuyến) -  $O(2^n)$ 
void towerOfHanoi(int n, char src, char aux, char dst) {
    if (n == 1) {
        printf("Chuyen dia 1 tu %c -> %c\n", src, dst);
        return;
    }
    towerOfHanoi(n - 1, src, dst, aux);
    printf("Chuyen dia %d tu %c -> %c\n", n, src, dst);
    towerOfHanoi(n - 1, aux, src, dst);
}

```

💡 Chứng minh độ phức tạp thời gian của Fibonacci đệ quy

Giả sử độ phức tạp khi tính $F(n)$ là $T(n) = O(a^n)$. Từ công thức đệ quy $F(n) = F(n-1) + F(n-2)$, ta có phương trình đặc trưng:

$$a^n = a^{n-1} + a^{n-2} \implies a^2 - a - 1 = 0 \quad (a > 0)$$

Nghiệm dương của phương trình là Tỷ lệ vàng:

$$\phi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

Vì vậy, độ phức tạp thời gian của Fibonacci đệ quy là $O(1.618^n)$ (mũ), cực kỳ kém hiệu quả khi n lớn.

Các Thuật Toán Tìm Kiếm (Search Algorithms)

Tìm Kiếm Tuyến Tính (Linear Search)

Thuật toán duyệt lần lượt qua các phần tử từ đầu đến cuối danh sách để tìm phần tử có khóa trùng khớp với giá trị cần tìm x .

- **Điều kiện áp dụng:** Dùng cho danh sách ngẫu nhiên (chưa sắp xếp) hoặc bất kỳ danh sách nào.
 - **Độ phức tạp:**
 - Tốt nhất: $O(1)$ (phần tử cần tìm nằm đầu mảng).
 - Xấu nhất: $O(n)$ (phần tử ở cuối mảng hoặc không có trong mảng).
 - Trung bình: $A(n) \approx \frac{n}{2} = O(n)$.
-

Tìm Kiếm Nhị Phân (Binary Search)

Thuật toán hoạt động theo nguyên lý chia để trị. So sánh khóa cần tìm x với phần tử ở giữa danh sách `middle`. * Nếu $x = a[middle]$: Tìm thấy. * Nếu $x < a[middle]$: Thu hẹp tìm kiếm ở nửa bên trái. * Nếu $x > a[middle]$: Thu hẹp tìm kiếm ở nửa bên phải.

- **Điều kiện áp dụng:** Danh sách bắt buộc phải được **sắp xếp (sorted)**.
 - **Độ phức tạp:**
 - Tốt nhất: $O(1)$ (phần tử cần tìm nằm ngay giữa).
 - Xấu nhất / Trung bình: $O(\log n)$.
-

Cài Đặt Tìm Kiếm Nhị Phân & Tuyến Tính (C)

```
// Tìm kiếm tuyến tính - O(n)
int linearSearch(int a[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x) return i; // Trả về chỉ mục
    }
    return -1; // Không tìm thấy
}
```

```
// Tìm kiếm nhị phân (Khử đệ quy) -  $O(\log n)$ 
int binarySearch(int a[], int n, int x) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Tránh tràn số nguyên
        if (a[mid] == x) return mid;
        else if (a[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

💡 Chứng minh độ phức tạp trung bình của Tìm kiếm nhị phân bằng cây quyết định

Xác suất tìm thấy x ở một vị trí bất kỳ là $p_i = 1/n$. Biểu diễn tìm kiếm nhị phân dưới dạng cây nhị phân, ở cấp độ thứ i có tối đa 2^{i-1} nút, thời gian thực hiện để chạm tới nút ở cấp i là i phép so sánh. Tổng số lần so sánh trung bình trên mảng kích thước n :

$$A(n) = \sum_{i=1}^k \frac{1}{n} \cdot 2^{i-1} \cdot i \approx \log_2 n + 1 - 0.5 \implies A(n) = O(\log n)$$

So với tìm kiếm tuyến tính $A(n) \approx n/2$, tìm kiếm nhị phân có hiệu suất vượt trội khi dữ liệu lớn.

Các Thuật Toán Sắp Xếp (Sorting Algorithms - I)

Sắp xếp là quá trình bố trí lại các phần tử dữ liệu trong danh sách theo một thứ tự xác định (tăng dần hoặc giảm dần). * **Sắp xếp nội (Internal sorting)**: Dữ liệu cần sắp xếp nằm hoàn toàn trong bộ nhớ RAM. * **Sắp xếp ngoài (External sorting)**: Dữ liệu quá lớn, phải lưu trữ và xử lý gián tiếp qua file trên bộ nhớ ngoài.

Các thuật toán phần I tập trung vào các thuật toán cơ bản có độ phức tạp thời gian trung bình $O(n^2)$.

Đổi Chỗ Trực Tiếp (Exchange/Interchange Sort)

Lấy một vị trí cố định i (từ đầu đến cuối mảng) làm mốc, so sánh $a[i]$ với tất cả các phần tử $a[j]$ đứng sau nó ($j > i$), hoán vị ngay lập tức khi phát hiện sai vị trí để đưa phần tử nhỏ nhất

về vị trí i .

Sắp Xếp Nổi Bọt (Bubble Sort)

Duyệt qua mảng nhiều lượt. Trong mỗi lượt, so sánh các cặp phần tử kế nhau $a[j]$ và $a[j + 1]$. Nếu sai thứ tự thì hoán vị. Sau mỗi lượt, phần tử lớn nhất sẽ “chìm” xuống cuối mảng.

Sắp Xếp Chọn (Selection Sort)

Tìm phần tử nhỏ nhất trong đoạn chưa được sắp xếp từ i đến $n - 1$, sau đó hoán vị nó với phần tử tại vị trí i . Khác với Interchange Sort, thuật toán này chỉ thực hiện tối đa 1 phép hoán vị sau mỗi vòng lặp lớn.

Sắp Xếp Chèn (Insertion Sort)

Xây dựng phân đoạn đã sắp xếp tăng dần từ đầu mảng. Tại mỗi bước, lấy phần tử kế tiếp chèn vào đúng vị trí thích hợp trong phân đoạn đã sắp xếp bằng cách dịch chuyển các phần tử lớn hơn về sau. * **Chèn nhị phân (Binary Insertion Sort)**: Sử dụng tìm kiếm nhị phân thay vì tìm kiếm tuyến tính để tìm vị trí chèn, giúp giảm số phép so sánh từ $O(n^2)$ xuống $O(n \log n)$ (tuy nhiên số phép dịch chuyển vẫn là $O(n^2)$).

Mã Nguồn C Các Thuật Toán Sắp Xếp

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// 1. Interchange Sort -  $O(n^2)$ 
void interchangeSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] > a[j]) swap(&a[i], &a[j]);
        }
    }
}
```



```

}

// 2. Bubble Sort -  $O(n^2)$ 
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) swap(&a[j], &a[j + 1]);
        }
    }
}

// 3. Selection Sort -  $O(n^2)$ 
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min_idx]) min_idx = j;
        }
        if (min_idx != i) swap(&a[i], &a[min_idx]);
    }
}

// 4. Insertion Sort -  $O(n^2)$ 
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

```

Danh Sách Liên Kết (Linked Lists)

Mảng có hạn chế lớn là kích thước cố định và chi phí chèn/xóa phần tử cao (do phải dịch chuyển phần tử). Danh sách liên kết khắc phục điều này bằng cách cấp phát động bộ nhớ cho

từng nút (node) riêng lẻ được liên kết với nhau qua các con trỏ.

Danh Sách Liên Kết Đơn (Singly Linked List)

Mỗi nút gồm: * **Data:** Dữ liệu cần lưu trữ (số, chuỗi, hoặc struct sinh viên...) * **Next:** Con trỏ trỏ đến nút kế tiếp trong danh sách. Nút cuối cùng có con trỏ **next = NULL**.

```
#include <stdlib.h>

// Định nghĩa Node
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Định nghĩa cấu trúc quản lý DSLK đơn
typedef struct LinkedList {
    Node* head;
} LinkedList;

// Tạo Node mới
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

Các Thao Tác Cơ Bản Trên DSLK Đơn

```
// Thêm vào đầu danh sách - O(1)
void insertHead(LinkedList* list, int value) {
    Node* newNode = createNode(value);
    newNode->next = list->head;
    list->head = newNode;
}

// Thêm vào cuối danh sách - O(n)
void insertTail(LinkedList* list, int value) {
    Node* newNode = createNode(value);
```

```

    if (list->head == NULL) {
        list->head = newNode;
        return;
    }
    Node* temp = list->head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Xóa nút đầu tiên - O(1)
void deleteHead(LinkedList* list) {
    if (list->head == NULL) return;
    Node* temp = list->head;
    list->head = list->head->next;
    free(temp);
}

// Xóa nút cuối cùng - O(n)
void deleteTail(LinkedList* list) {
    if (list->head == NULL) return;
    if (list->head->next == NULL) {
        free(list->head);
        list->head = NULL;
        return;
    }
    Node* temp = list->head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

// Duyệt và in danh sách - O(n)
void printList(LinkedList* list) {
    Node* temp = list->head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
}

```

```
printf("NULL\n");  
}
```

Các Loại Danh Sách Liên Kết Khác

Danh Sách Liên Kết Đôi (Doubly Linked List)

Mỗi nút chứa hai con trỏ: * **next**: Trỏ đến nút kế tiếp. * **prev**: Trỏ đến nút phía trước. * *Ưu điểm*: Có thể duyệt danh sách theo cả hai chiều một cách dễ dàng. Thao tác xóa nút khi đã biết con trỏ nút đó tốn thời gian $O(1)$.

Danh Sách Liên Kết Vòng (Circular Linked List)

Nút cuối cùng của danh sách thay vì trỏ đến NULL sẽ trỏ ngược về nút đầu tiên (**head**). Giúp việc duyệt vòng lặp vô hạn qua các phần tử đơn giản hơn.

Ngăn Xếp (Stack)

Định Nghĩa & Nguyên Lý LIFO

Ngăn xếp là cấu trúc danh sách tuyến tính hoạt động theo nguyên lý **Vào Sau Ra Trước - LIFO (Last In First Out)**. Thao tác thêm (**push**) và xóa (**pop**) chỉ thực hiện ở duy nhất một đầu gọi là đỉnh ngăn xếp (**top**).

Cài Đặt Stack

Cài Đặt Bằng Mảng (Array-based Stack)

- Sử dụng biến chỉ mục **top** (bắt đầu từ -1 khi rỗng).
- Hạn chế*: Kích thước tối đa của ngăn xếp (**MAX**) phải khai báo trước.

```

#define MAX 100

typedef struct StackArray {
    int data[MAX];
    int top;
} StackArray;

void initStackArray(StackArray* s) { s->top = -1; }
int isEmptyArray(StackArray* s) { return s->top == -1; }
int isFullArray(StackArray* s) { return s->top == MAX - 1; }

void pushArray(StackArray* s, int val) {
    if (isFullArray(s)) return;
    s->data[++(s->top)] = val;
}

int popArray(StackArray* s) {
    if (isEmptyArray(s)) return -1;
    return s->data[(s->top)--];
}

```

Cài Đặt Bằng Danh Sách Liên Kết (Linked-list Stack)

- Không giới hạn kích thước động. Các thao tác push và pop thực hiện tương ứng với insertHead và deleteHead trên DSLK đơn.
- *Độ phức tạp*: Lưu trữ $O(n)$, các phép toán thêm/xóa $O(1)$.

```

typedef struct StackNode {
    int data;
    struct StackNode* next;
} StackNode;

typedef struct StackList {
    StackNode* top;
} StackList;

void initStackList(StackList* s) { s->top = NULL; }
int isEmptyList(StackList* s) { return s->top == NULL; }

void pushList(StackList* s, int val) {
    StackNode* newNode = (StackNode*)malloc(sizeof(StackNode));

```

```

    newNode->data = val;
    newNode->next = s->top;
    s->top = newNode;
}

int popList(StackList* s) {
    if (isEmptyList(s)) return -1;
    StackNode* temp = s->top;
    int val = temp->data;
    s->top = s->top->next;
    free(temp);
    return val;
}

```

Ứng Dụng Của Stack

1. **Kiểm tra dấu ngoặc hợp lệ (Parentheses Checker):** Quét biểu thức, đẩy ngoặc mở (, {, [vào stack. Gặp ngoặc đóng thì kiểm tra so khớp với ngoặc ở top stack và pop ra. Nếu kết thúc chuỗi mà stack rỗng thì hợp lệ.
2. **Đảo ngược chuỗi (Reversing a string):** Đẩy lần lượt ký tự vào stack, sau đó pop ra đến khi rỗng.
3. **Ký pháp Ba Lan & Đánh giá Biểu thức:**
 - **Infix (Trung tố):** Toán tử nằm giữa các toán hạng (ví dụ: $a + b$).
 - **Prefix (Tiền tố):** Toán tử ở trước (ví dụ: $+ab$).
 - **Postfix (Hậu tố):** Toán tử ở sau (ví dụ: $ab+$).

Thuật Toán Chuyển Infix Sang Postfix (Shunting Yard của Dijkstra)

Quét biểu thức Infix từ trái qua phải: * Nếu là **toán hạng (số)**: Đưa thẳng vào kết quả Postfix. * Nếu là **ngoặc mở (**: Đẩy vào Stack. * Nếu là **ngoặc đóng)**: Pop lần lượt các toán tử từ Stack ra đưa vào Postfix cho đến khi gặp ngoặc mở (. Xóa bỏ cặp ngoặc. * Nếu là **toán tử**: So sánh độ ưu tiên với phần tử ở đỉnh Stack. Nếu độ ưu tiên của toán tử hiện tại nhỏ hơn hoặc bằng toán tử ở đỉnh Stack, pop toán tử ở đỉnh ra đưa vào Postfix, sau đó push toán tử hiện tại vào Stack.

Hàng Đợi (Queue)

Định Nghĩa & Nguyên Lý FIFO

Hàng đợi là cấu trúc tuyến tính hoạt động theo nguyên lý **Vào Trước Ra Trước - FIFO (First In First Out)**. * **Enqueue**: Thêm phần tử vào cuối hàng đợi (**rear**). * **Dequeue**: Lấy phần tử ra khỏi đầu hàng đợi (**front**).

Hàng Đợi Vòng Bằng Mảng (Circular Array Queue)

Nếu dùng mảng tuyến tính thông thường, sau nhiều lần **enqueue** và **dequeue**, các con trỏ **front** và **rear** tịnh tiến về cuối mảng làm lãng phí không gian trống ở phía trước. **Circular Queue** khắc phục điều này bằng cách xoay vòng chỉ mục thông qua toán tử chia lấy dư (%).

```
#define QUEUE_MAX 5

typedef struct CircularQueue {
    int data[QUEUE_MAX];
    int front;
    int rear;
} CircularQueue;

void initQueue(CircularQueue* q) {
    q->front = -1;
    q->rear = -1;
}

int isFull(CircularQueue* q) {
    return (q->front == 0 && q->rear == QUEUE_MAX - 1) || (q->front == q->rear + 1);
}

int isEmpty(CircularQueue* q) {
    return q->front == -1;
}

void enqueue(CircularQueue* q, int val) {
    if (isFull(q)) return;
    if (isEmpty(q)) {
        q->front = 0;
    }
}
```

```

        q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % QUEUE_MAX; // Xoay vòng
    }
    q->data[q->rear] = val;
}

int dequeue(CircularQueue* q) {
    if (isEmpty(q)) return -1;
    int val = q->data[q->front];
    if (q->front == q->rear) { // Chỉ còn 1 phần tử
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % QUEUE_MAX;
    }
    return val;
}

```

Cây Tìm Kiếm Nhị Phân (Binary Search Tree - BST)

Các Khái Niệm Về Cây

- **Cây (Tree):** Cấu trúc phi tuyến gồm một nút gốc (root) liên kết với tập hợp các cây con không chu trình.
- **Chiều sâu (Depth) của nút:** Số lượng cạnh từ gốc đến nút đó.
- **Bậc (Degree) của nút:** Số lượng nút con của nút đó.
- **Nút lá (Leaf):** Nút có bậc bằng 0 (không có con).

Cây Nhị Phân (Binary Tree)

Cây nhị phân là cây có bậc tối đa bằng 2 (mỗi nút chỉ có tối đa 2 con: con trái và con phải).

Các Phương Pháp Duyệt Cây Nhị Phân

Duyệt đệ quy theo các trật tự: * **Duyệt trước (Pre-order - NLR)**: Nút gốc → Cây con Trái → Cây con Phải. * **Duyệt giữa (In-order - LNR)**: Cây con Trái → Nút gốc → Cây con Phải. * **Duyệt sau (Post-order - LRN)**: Cây con Trái → Cây con Phải → Nút gốc.

Cây Tìm Kiếm Nhị Phân (BST)

BST là cây nhị phân thỏa mãn điều kiện: Với mọi nút bất kỳ trên cây, giá trị của các nút con trái luôn nhỏ hơn nút gốc, và giá trị của các nút con phải luôn lớn hơn nút gốc.

Tip

Duyệt cây BST theo thứ tự **In-order (LNR)** sẽ cho ra một dãy giá trị sắp xếp **tăng dần**.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct BSTNode {
    int key;
    struct BSTNode* left;
    struct BSTNode* right;
} BSTNode;

BSTNode* createBSTNode(int value) {
    BSTNode* newNode = (BSTNode*)malloc(sizeof(BSTNode));
    newNode->key = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

Các Thao Tác Trên BST (Tìm kiếm, Thêm, Xóa)

```
// 1. Tìm kiếm trên BST - O(log n)
BSTNode* search(BSTNode* root, int key) {
```

```

    if (root == NULL || root->key == key) return root;
    if (key < root->key) return search(root->left, key);
    return search(root->right, key);
}

// 2. Thêm một khóa mới vào BST -  $O(\log n)$ 
BSTNode* insert(BSTNode* root, int key) {
    if (root == NULL) return createBSTNode(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}

// Tìm nút cực trái của cây con (Nút nhỏ nhất)
BSTNode* minValueNode(BSTNode* node) {
    BSTNode* current = node;
    while (current && current->left != NULL) current = current->left;
    return current;
}

// 3. Xóa một nút khỏi BST -  $O(\log n)$ 
BSTNode* deleteNode(BSTNode* root, int key) {
    if (root == NULL) return root;
    if (key < root->key) root->left = deleteNode(root->left, key);
    else if (key > root->key) root->right = deleteNode(root->right, key);
    else { // Tìm thấy nút cần xóa
        // Trường hợp 0 hoặc 1 con
        if (root->left == NULL) {
            BSTNode* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            BSTNode* temp = root->left;
            free(root);
            return temp;
        }
        // Trường hợp 2 con: Tìm nút thế mạng leftmost của cây con phải
        BSTNode* temp = minValueNode(root->right);
        root->key = temp->key; // Sao chép giá trị thế mạng lên
        root->right = deleteNode(root->right, temp->key); // Xóa nút thế mạng cũ
    }
    return root;
}

```

}

⚠ Sự suy biến của BST

Nếu ta chèn vào cây BST một dãy các số đã được sắp xếp tăng dần hoặc giảm dần (ví dụ: 10, 20, 30, 40), cây nhị phân sẽ suy biến thành một **Danh sách liên kết tuyến tính (độ lệch hoàn toàn về một phía)**. Khi đó, độ phức tạp của thao tác tìm kiếm, chèn, xóa sẽ tăng từ $O(\log n)$ lên $O(n)$. Để giải quyết vấn đề này cần cấu trúc cây tự cân bằng (như cây AVL hoặc cây Đỏ-Đen).

Bảng Băm (Hash Table)

Kỹ Thuật Băm (Hashing)

Bảng băm là cấu trúc dữ liệu ánh xạ trực tiếp một giá trị khóa (**key**) thành chỉ mục của mảng (**index**) thông qua **hàm băm (hash function)**. Mục tiêu là cho phép thực hiện các thao tác Tìm kiếm, Chèn, Xóa với độ phức tạp lý tưởng $O(1)$.

Các Hàm Băm Phổ Biến

1. Phương pháp chia (Division method):

$$h(k) = k \pmod{M}$$

Với M là kích thước của bảng băm (nên chọn là số nguyên tố để tối ưu phân phối).

2. Phương pháp nhân (Multiplication method):

$$h(k) = \lfloor M \cdot (k \cdot A \pmod{1}) \rfloor$$

Với hằng số $A \approx \frac{\sqrt{5}-1}{2} \approx 0.618$ (Tỷ lệ vàng).

3. Phương pháp bình phương (Mid-Square method):

Bình phương khóa k^2 rồi trích xuất một số chữ số ở giữa để làm index.

4. Phương pháp gấp (Folding method):

Chia khóa k thành nhiều phần bằng nhau rồi cộng các phần lại.

Giải Quyết Xung Đột (Collision Resolution)

Xung đột xảy ra khi hai khóa khác nhau qua hàm băm cho ra cùng một chỉ mục ($h(k_1) = h(k_2)$).

Cấp Địa Chỉ Mở (Open Addressing)

Khi xảy ra xung đột, thuật toán dò tìm một vị trí trống khác trong bảng băm. * **Dò tìm tuyến tính (Linear probing)**: Dò tuần tự từng ô kế tiếp:

$$h(k, i) = (h'(k) + i) \pmod{M}$$

Hạn chế: Gây ra hiện tượng Primary Clustering (các phần tử xung đột tập trung dồn cục lại một khu vực làm chậm tìm kiếm). * **Dò tìm bậc hai (Quadratic probing)**: Dò nhảy bước theo bình phương:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod{M}$$

* **Băm kép (Double Hashing)**: Sử dụng hàm băm thứ hai để quyết định bước nhảy:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{M}$$

* **Rehashing (Băm lại)**: Khi bảng băm đạt ngưỡng tải (gần đầy), khởi tạo bảng băm mới có kích thước gấp đôi và thực hiện băm lại toàn bộ phần tử sang bảng mới.

Phân Mạch Liên Kết (Chaining)

Mỗi ô trong bảng băm thay vì lưu trực tiếp phần tử sẽ lưu một con trỏ tới một **Danh sách liên kết**. Các phần tử có cùng chỉ số băm sẽ được nối thêm vào danh sách liên kết của ô đó. Thao tác chèn có độ phức tạp $O(1)$, thao tác tìm kiếm phụ thuộc vào độ dài trung bình của các liên kết.

```
#include <stdio.h>
#include <stdlib.h>

#define HASH_SIZE 10

typedef struct HashNode {
    int key;
    int val;
    struct HashNode* next;
} HashNode;

typedef struct HashTable {
```

```

    HashNode* buckets[HASH_SIZE];
} HashTable;

int hashFunction(int key) {
    return key % HASH_SIZE;
}

void initHashTable(HashTable* ht) {
    for (int i = 0; i < HASH_SIZE; i++) ht->buckets[i] = NULL;
}

// Chèn phần tử vào bảng băm (Chaining) - O(1)
void hashInsert(HashTable* ht, int key, int val) {
    int idx = hashFunction(key);
    HashNode* newNode = (HashNode*)malloc(sizeof(HashNode));
    newNode->key = key;
    newNode->val = val;
    newNode->next = ht->buckets[idx];
    ht->buckets[idx] = newNode;
}

// Tìm kiếm trên bảng băm - O(1) trung bình
int hashSearch(HashTable* ht, int key) {
    int idx = hashFunction(key);
    HashNode* temp = ht->buckets[idx];
    while (temp != NULL) {
        if (temp->key == key) return temp->val;
        temp = temp->next;
    }
    return -1; // Không tìm thấy
}

```